

Kalman Filters

Arin Ghazarian

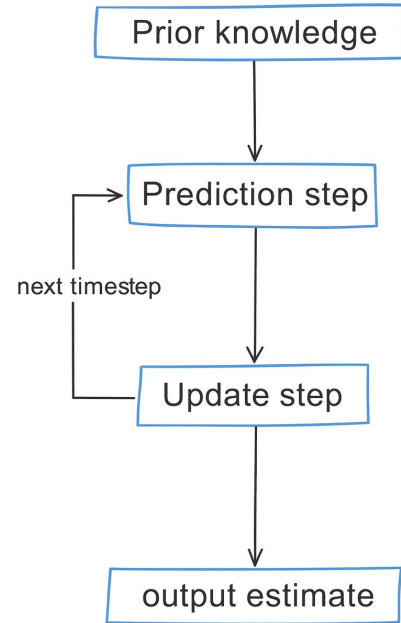
California State University Long Beach

Motivation

- We often have to estimate a quantity that changes over time but measurements are noisy.
- Examples:
 - Tracking objects in video
 - Estimating robot position
 - Smoothing noisy sensor data

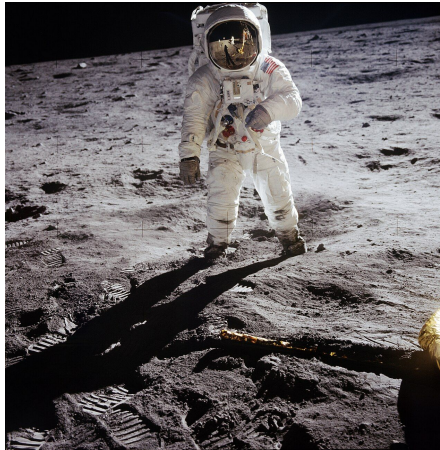
What is a Kalman Filter?

- The Kalman filter is an **efficient recursive filter** estimating the internal state of a **linear dynamic system** from a series of **noisy measurements**.
- A Kalman Filter is an algorithm that estimates the true state of a dynamic system from noisy observations.
- It combines predictions from a model with actual measurements to produce optimal estimates.
- Understanding its core principles is essential for anyone working with robotics, drones, or other intelligent autonomous technologies.

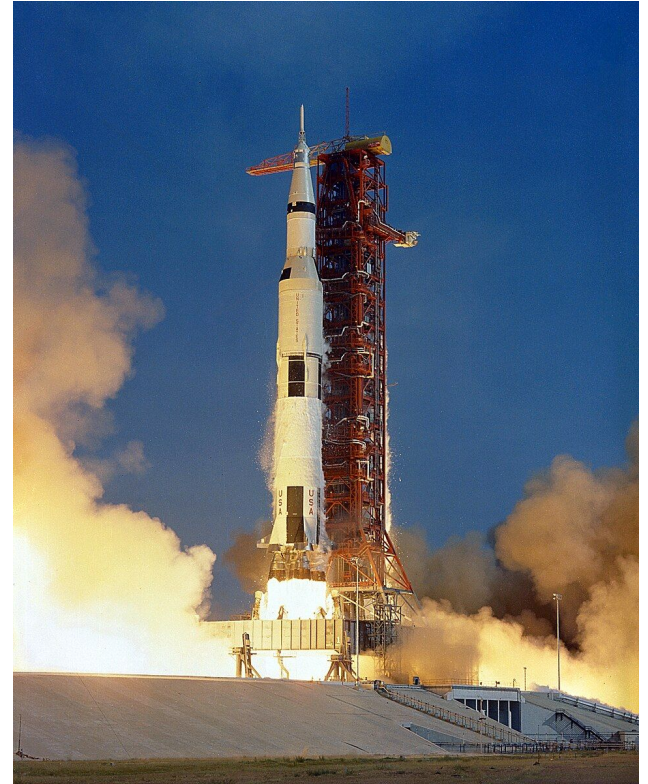


What is a Kalman Filter?

- [R.E. Kalman, 1960: A New Approach to Linear Filtering and Prediction Problems.](#)
- The Kalman Filter has been around since the 1960s and was even used in NASA's Apollo program, making it a foundational concept in autonomous systems.



[source](#)



Core Idea

- Predict what the next state should be.
- Compare prediction with noisy observation.
- Adjust the estimate to be somewhere between the two, depending on uncertainty.

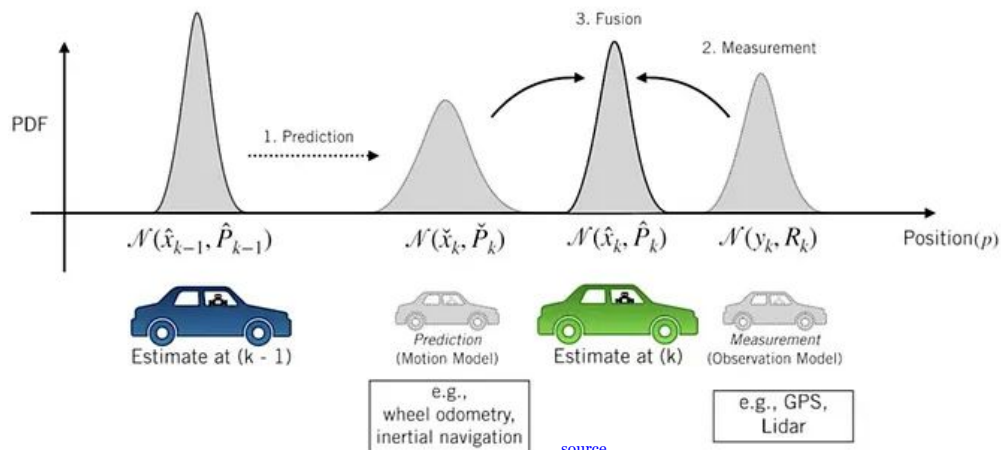
Kalman Filter

- Estimates the state of a linear system affected by stochastic (random) processes, ideally assuming **Gaussian noise** for optimal performance.
- Based on **Bayes' rule**, combining prior knowledge with new measurements to update the belief about the system's state.
- Assumes a **Markov property**, meaning it only needs the current state to predict the next—no need for full historical data.
- **Recursive** in nature, reusing its previous output as input for the next estimation step.

Simple Example

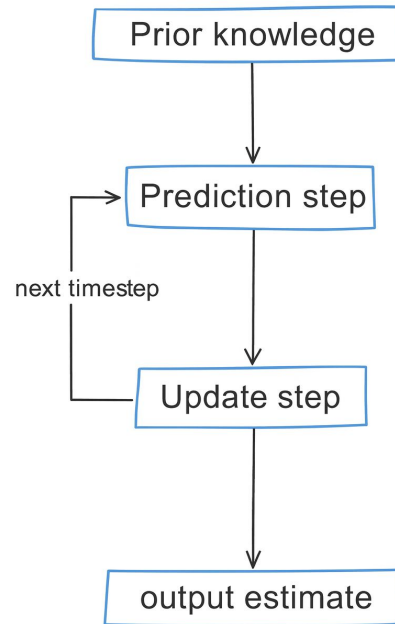
- Imagine tracking a moving car using a noisy GPS.
- We predict where the car should be based on its motion, then correct that prediction using the new GPS reading.

The Kalman Filter I Prediction and Correction



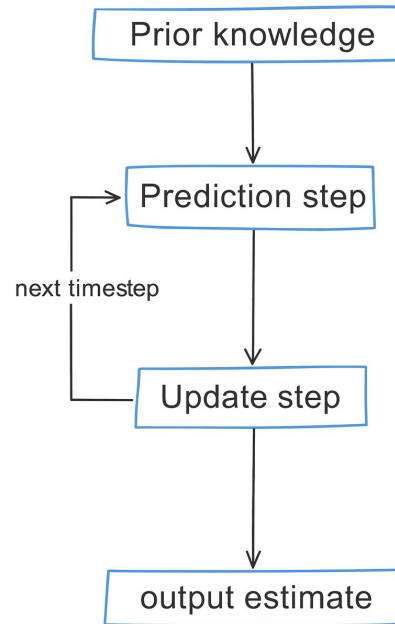
Kalman Filter

- Filter: An algorithm that receives input signals—typically from sensors—and processes them to produce a refined or transformed output.
- A Kalman Filter is a special type of filter:
 - Recursive
 - Computationally efficient
 - Designed to estimate the state of a dynamic system using a series of noisy measurements
- Kalman Filter is an iterative estimation algorithm that combines prior knowledge and new observations to produce an optimal estimate of the underlying variables, even when the data is uncertain or noisy.



Kalman Filter

- Iterative: Uses only the previous estimate and the latest measurement—no need to store all past data (Markov Process)
- Efficient: Avoids heavy computations, simply updates the current estimate as new data arrives
- Optimal (in MSE sense): Produces an estimate that minimizes mean-squared error (MSE)



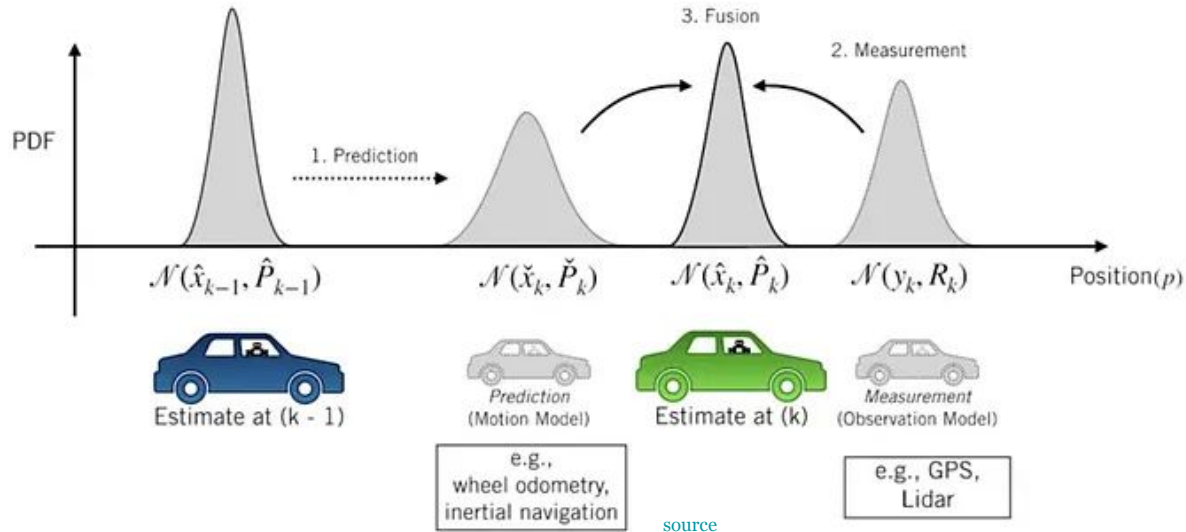
Iterations of Prediction/Correction

- Kalman filter operates by performing the following in iterations:
 - Predicting the new state and its uncertainty using the motion model
 - Correcting with the new observation measurement



Prediction and Correction

The Kalman Filter I Prediction and Correction

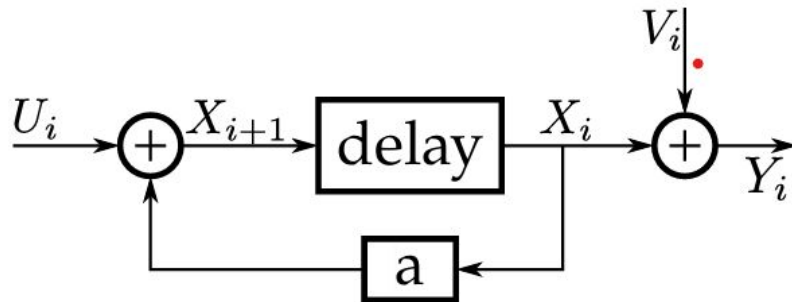


One-dimensional System

- Our objective is to estimate X_i given Y_i and all past measurements of Y (that is, $Y_{i-1}, Y_{i-2}, \dots, Y_0$).
- At each time step, the input U_i is added to X_i to get the new value for X . This is an input which changes the actual state of the system.
- V_i is noise that gets added to the true value to in the measurement process. This does not change the state of the system; it merely corrupts the measurement.
- We will assume that U and V are zero-mean independent white noise processes. That is, each U_i is unrelated to any other U_i and $E[U_i] = 0$.

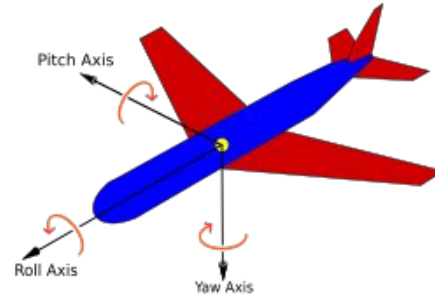
$$X_{i+1} = aX_i + U_i$$

$$Y_i = X_i + V_i$$

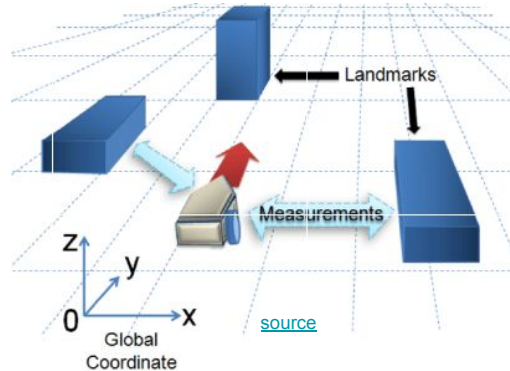


Application

- Broadly, it's useful for any type of tracking application, such as:
 - Tracking missiles
 - Estimating position of aircraft
 - Surveillance of highway traffic
 - GPS-based motion estimation
 - Economics applications (estimating demand for international reserves)
 - Mobile robot localization
 - Sensor fusion (e.g., IMU + GPS)



[source](#)



[source](#)



[source](#)

System Model

- We define:
 - State vector x_k = system variables (e.g., position, velocity)
 - Transition model: $x_k = F \cdot x_{k-1} + w_k$
 - Observation model: $z_k = H \cdot x_k + v_k$
 - w_k, v_k are process and measurement noise.

Prediction Step

- Predict the next state and its uncertainty:

State prediction: $\hat{\mathbf{x}}_{k+1|k} = \mathbf{F} \cdot \hat{\mathbf{x}}_{k|k}$

Covariance prediction: $\mathbf{P}_{k+1|k} = \mathbf{F} \cdot \mathbf{P}_{k|k} \cdot \mathbf{F}^T + \mathbf{Q}$

$\mathbf{Q}$ is process noise and $\mathbf{R}$ is measurement noise

Update Step

- When a new measurement z_k arrives:
- Compute Kalman Gain:
 - $K_k = P_k | k-1 \cdot H^T \cdot (H \cdot P_k | k-1 \cdot H^T + R)^{-1}$
- Update estimate:
 - $\hat{x}_k | k = \hat{x}_k | k-1 + K_k (z_k - H \cdot \hat{x}_k | k-1)$
- Update covariance:
 - $P_k | k = (I - K_k H) P_k | k-1$

Intuition Behind Kalman Gain

- Kalman Gain K decides how much we trust the measurement vs. our model.
 - Large $K \rightarrow$ trust measurement more.
 - Small $K \rightarrow$ trust model more.
- Balance depends on uncertainties R and Q .

Assumptions

- System dynamics are linear.
- Noise is Gaussian (zero mean, known covariance).
- Process and measurement noises are independent.

Extended Kalman Filter (EKF)

- For nonlinear systems
 - Linearize the system around current estimate using Jacobians.
 - Update as in Kalman filter but with local linear approximations.

Unscented Kalman Filter (UKF)

- Instead of linearizing, UKF uses sigma points to capture mean and covariance under nonlinear transformations.
- More accurate than EKF for strong nonlinearities.

Comparison EKF vs UKF

- EKF: simpler, but may diverge if linearization is poor.
- UKF: more accurate, computationally heavier.



<https://www.youtube.com/watch?v=bm3cwEP2nUo>



<https://www.youtube.com/watch?v=IFeClbljreY>

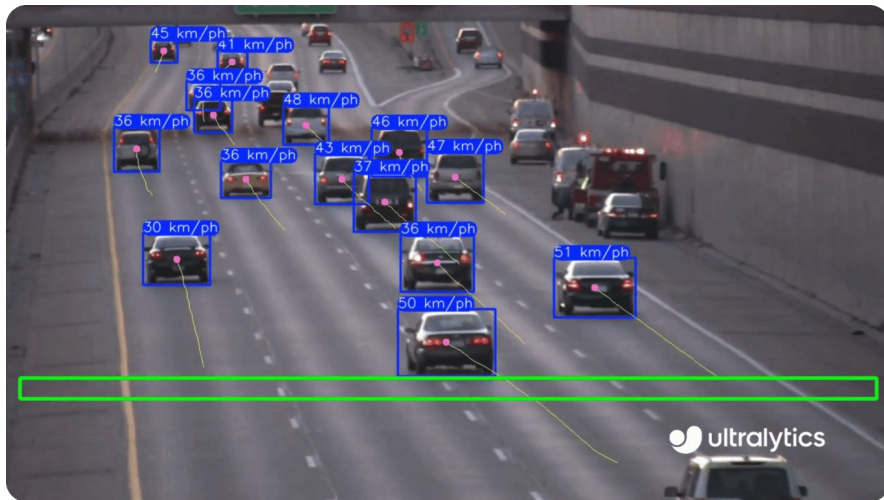


https://www.youtube.com/watch?v=s_9lnuQAx-g

Case Study 1: Object Tracking in a Video

Object Tracking

- Goal: Track moving objects (e.g., people, cars) in video using Kalman filtering.
- Each object has a state $[x, y, v_x, v_y]$.
- Measurements come from detector bounding boxes.



Tracking Example

- At each frame:
 - predict object position using velocity.
 - Receive detection (e.g., YOLO box center).
 - Update the prediction.
 - Output filtered position for smoother tracking.

State Vector for Tracking

- $x = [x_position, y_position, x_velocity, y_velocity]^T$
- F = motion model matrix
- $z = [x_measured, y_measured]^T$
- H = measurement model matrix

Noise Covariance Tuning

- Q (process noise): controls how much we trust model dynamics.
- R (measurement noise): controls how much we trust sensor readings.
- Choosing them well is crucial for performance.

Advantages in Tracking

- Smooths noisy detections.
- Can predict where an object should appear even if temporarily missed.
- Efficient: runs in real-time.



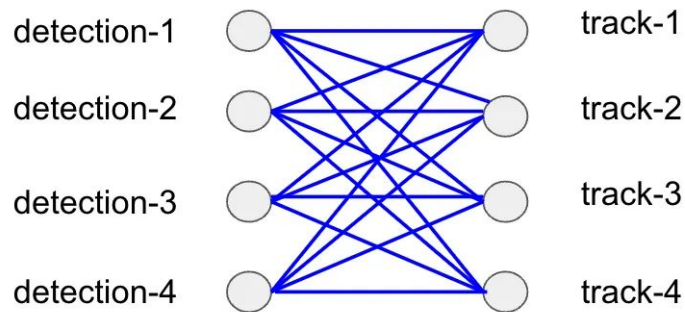
Multiple Object Tracking: Assignment & Kalman Filter

- Each tracked object maintains a Kalman filter that predicts its next state (position, velocity, bounding box, etc.).
- At every frame, the Kalman filter generates a predicted state and uncertainty (covariance) for each track.
- New detections are assigned to existing tracks by computing a cost between predictions and detections.
- Common assignment costs include:
 - Mahalanobis distance (uses Kalman covariance – uncertainty-aware)
 - Euclidean distance (position-only)
 - IoU-based distance (bounding box overlap)

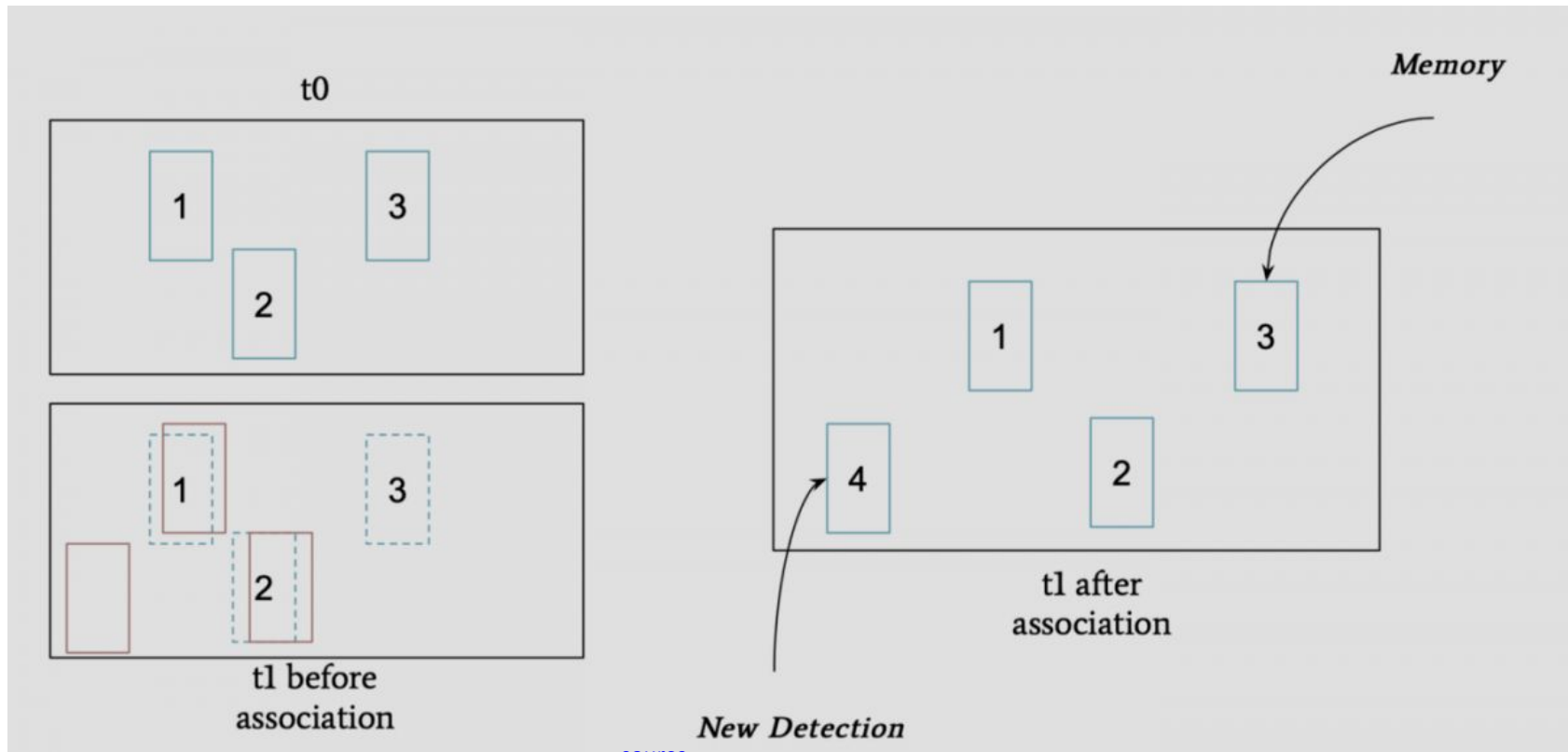
Multiple Object Tracking: Assignment & Kalman Filter

- Unmatched detections start new tracks; unmatched tracks are marked lost, but Kalman prediction continues.
- The Kalman filter update step refines each track using its assigned detection; unassigned tracks rely on prediction only.
- The Hungarian algorithm is typically used to find the optimal one-to-one assignment that minimizes total cost.

	Track-1	Track-2	Track-3	Track-4
Detection-1	80	20	35	65
Detection-2	28	75	32	55
Detection-3	67	12	76	27
Detection-4	35	56	43	90



Multiple Object Tracking: Assignment & Kalman Filter



Challenges

- Requires correct initialization.
- Sensitive to wrong noise covariances Q and R .
- Cannot handle sudden motion changes well (nonlinear behavior).

Demo: Tracking the Ball

- https://colab.research.google.com/github/Acytoo/Kalman-Filter-Multiple-Object-Tracking/blob/main/Kalman_Filter_Multiple_Object_Tracking.ipynb

Case Study 2: Drone Pose Estimation

Pose Estimation in CrazyFLie

- Principles of Kalman filtering and how it is applied within drone firmware



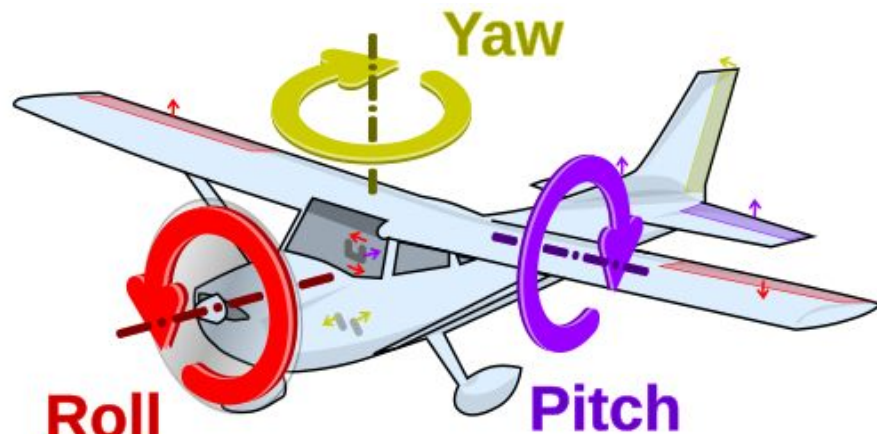
IMU



- Components
 - Accelerometers: Measure linear acceleration
 - Gyroscopes: Measure angular velocity (rate of rotation).
 - Measure magnetic field strength, helping to determine absolute heading (north).
- Relying solely on the IMU causes position drift over time. For improved stability and absolute positioning, the IMU data is often combined with external positioning systems (like motion capture systems for indoor flight or GPS for outdoor flight) or additional sensors such as an optical flow sensor and Time-of-Flight (ToF) distance sensor (e.g., via a Flow deck).

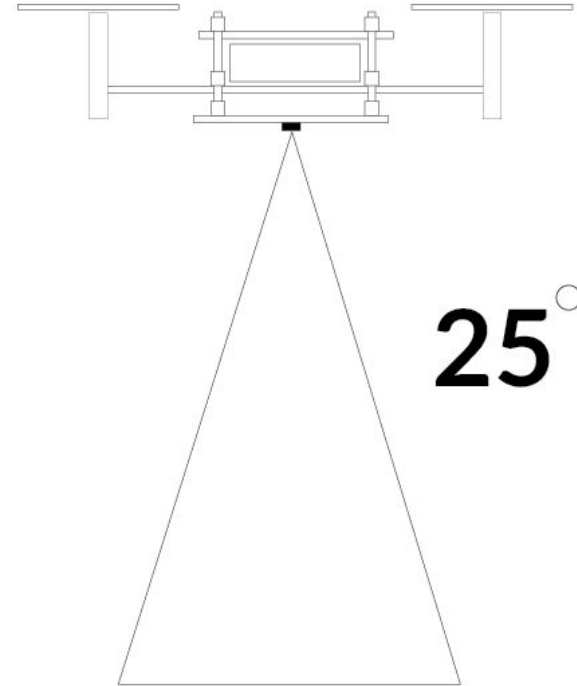
IMU

- IMUs work, in part, by detecting changes in pitch, roll, and yaw
- Drift: An IMU provides acceleration and angular rate data. To get position, these measurements must be integrated twice with respect to time.
 - Any small error or bias in the initial measurements, or the integration process, accumulates rapidly, causing the estimated position to "drift" significantly from the true location.
 - Noise: Consumer-grade IMUs are particularly noisy, making the drift problem worse.
 - Gravity: Accelerometers measure the combined force of motion and gravity. Distinguishing the direction of gravity from actual linear acceleration [can](#) be tricky, especially during dynamic movements.



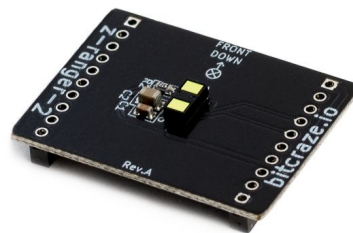
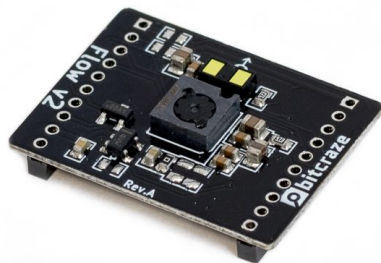
Z-ranger deck

- The range sensor measures distance using a cone-shaped detection area, reporting the closest object within that cone rather than a single point.
- At higher altitudes the cone covers a larger area, so near walls or obstacles the sensor may incorrectly report their distance instead of the floor.



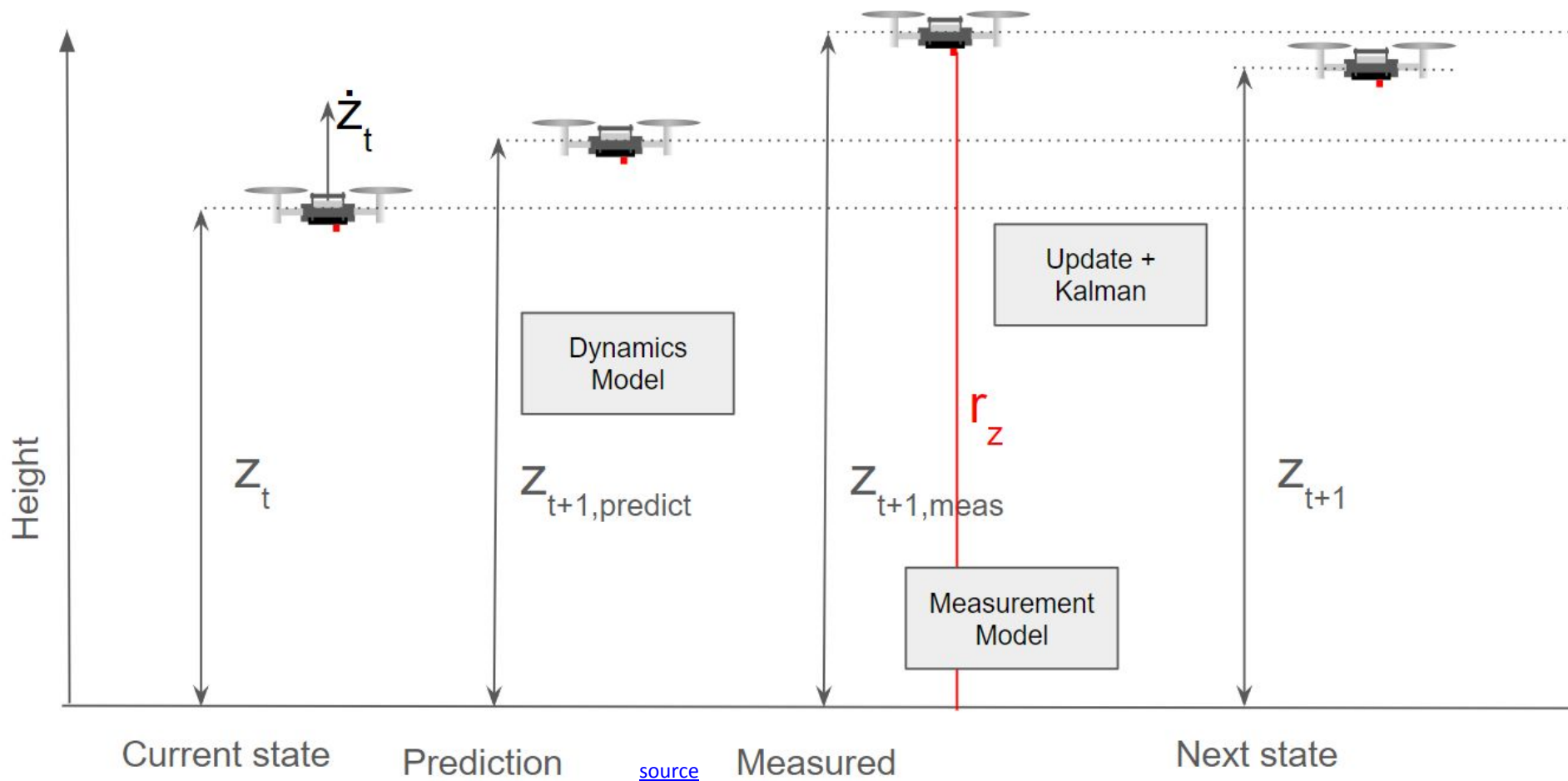
Deck sensors

- Downward-facing time-of-flight sensors
 - Flow deck
 - Z-ranger deck



Data fusion

- This raw distance measurement is combined with data from the Crazyflie's Inertial Measurement Unit (IMU) through an Extended Kalman Filter (EKF) to produce a more accurate and stable estimate of its position and altitude.



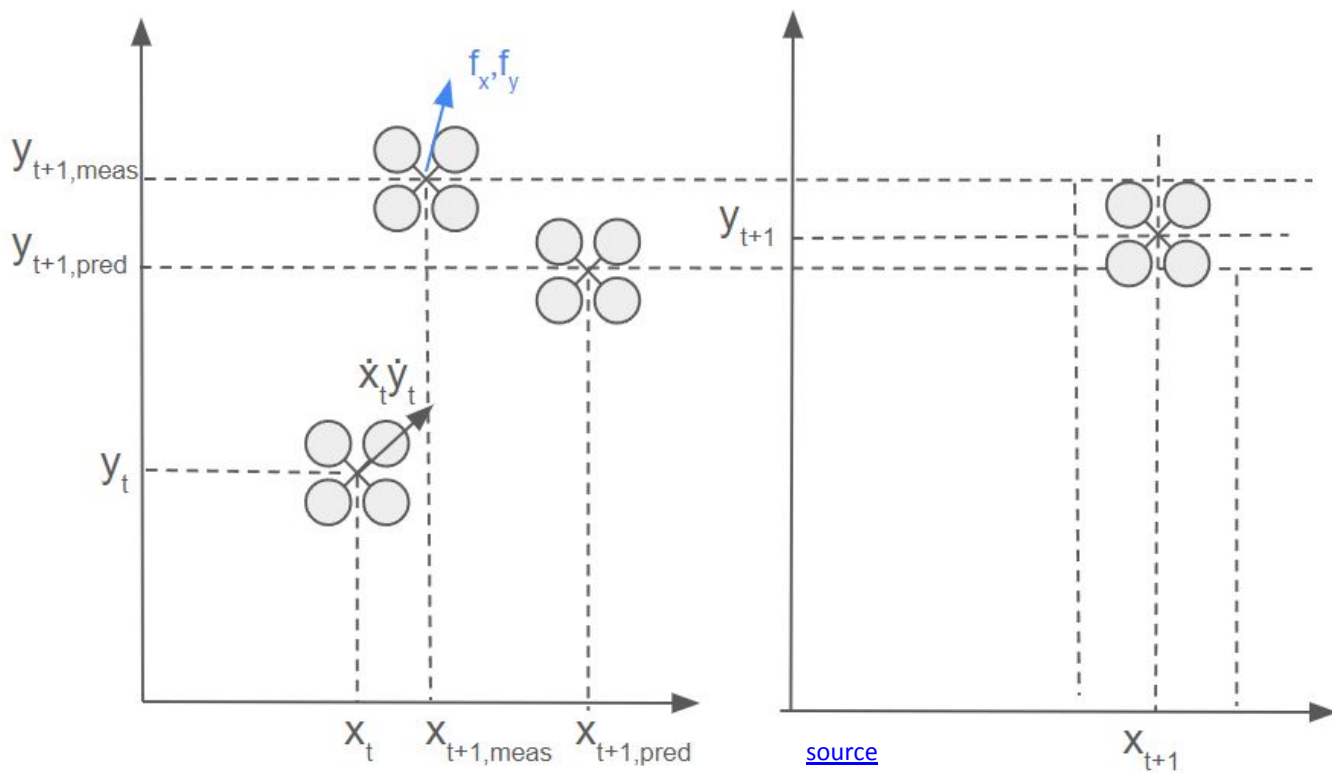
Drone— Height Estimation

- The drone starts with its current estimated height z_t and vertical velocity $\dot{z}_t$
- In the Prediction Phase, it predicts the next height as $\hat{z}_{t+1}^{\text{predict}}$

$$\hat{z}_{t+1}^{\text{predict}} = z_t + \dot{z}_t$$

- In the Update (Correction) Phase, a height measurement from a range sensor is obtained and converted into the estimated measurement z_{t+1}^{meas}
 - a. The filter calculates the Kalman Gain using the predicted and measurement covariances, and combines $\hat{z}_{t+1}^{\text{predict}}$ and z_{t+1}^{meas} to produce an improved estimate $\hat{z}_{t+1}$
- If the sensor is more accurate (e.g., infrared range sensor), the estimate leans closer to the measurement; if sensor is noisy (e.g., ultrasonic or barometer), it relies more on the prediction from the dynamics model

2D Example



Extended Kalman Filter in CrazyFlie Firmware

- Initialize the state and variances `kalmanCoreInit()` in `kalman_core.c`
- Prediction step with the dynamics model: `predictDt()` in `kalman_core.c`
- Innovation and update of the covariance with the measurement update: `kalmanCoreScalarUpdate()` in `kalman_core.c`
 - All measurement models can be found in separate files in the `kalman_core/` folder
 - The height measurement model for TOF range sensor like in the 1D example: `kalmanCoreUpdatewithToF()` in `mm_tof.c`
 - The flow measurement model for the flow sensor like in the 2D example: `kalmanCoreUpdateWithFlow()` in `mm_flow.c`
- Finalizing the state (by rotating all the state variables in the correct orientation: `kalmanCoreFinalize()` in `kalman_core.c`

Summary

- Kalman filter fuses prediction and observation optimally under uncertainty.
- Simple, elegant, and powerful.
- Foundation for modern tracking and state estimation systems.